

软件测试

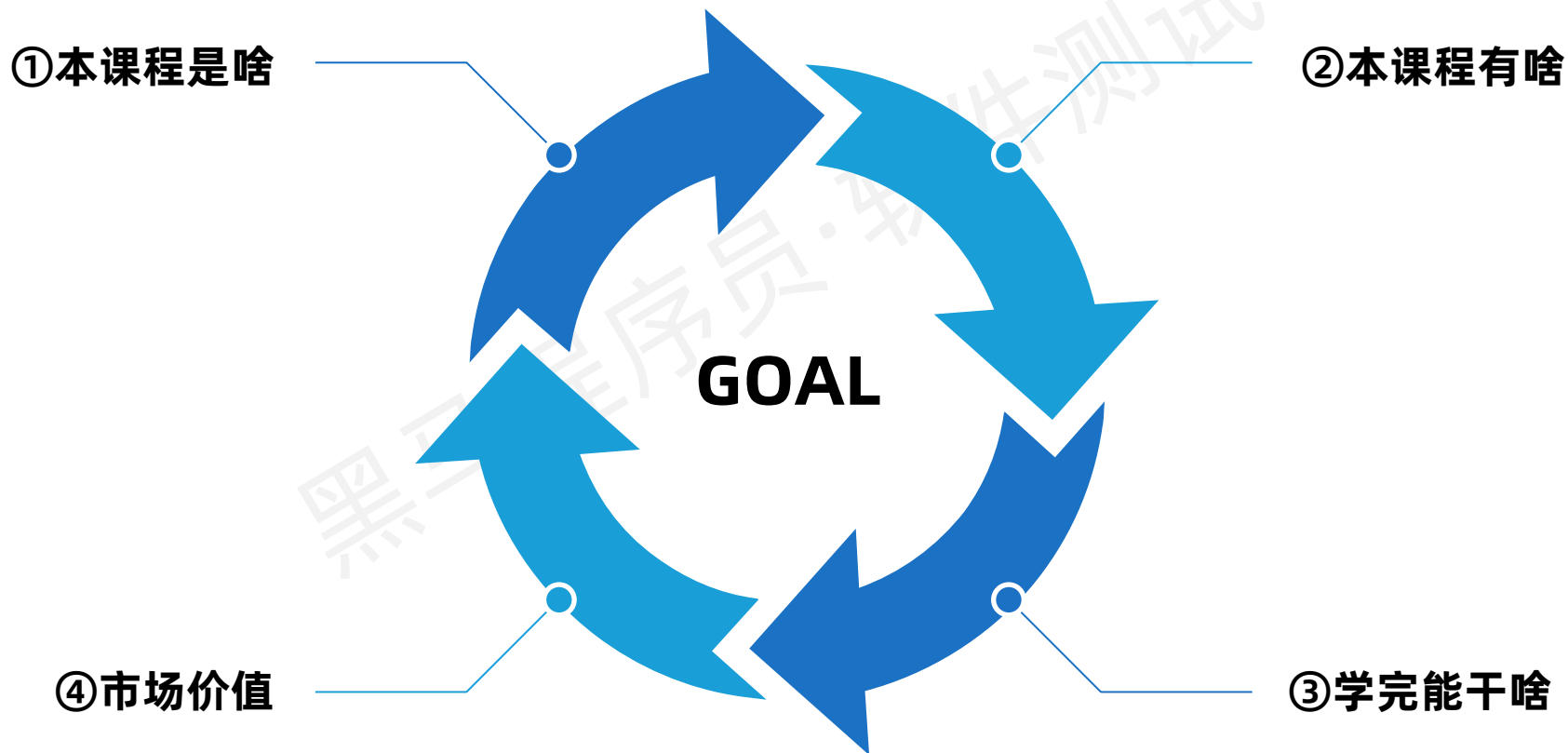
APP手工测试



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

本阶段课程介绍



本课程是啥

1. 本课程是软件测试之APP**功能**及**非功能**测试课，主要基于**有初步测试基础**的软件测试人员进行开展大项目全流程测试设计的课程。
2. 面向群体
 - ① 已经学完测试基础课程的准测试人员

本课程有啥



学完能干啥



- ① 具备APP系统的**功能及非功测试**的能力
- ② 具备相似类项目的测试**设计思维**

市场价值

移动端测试 软件测试

11-15K

五险一金

❤ 感兴趣

立即沟通

查看所有职位

- 1、负责Android及iOS端APP的功能测试、性能测试、自动化测试、兼容性测试
- 2、维护、开发自动化测试脚本，执行测试用例，分析测试结果
- 3、持续改进自动化测试流程和测试方法，提高测试效率
- 4、不断学习和积累项目经验，从而提高测试技能、优化测试方法、提高产品质量
- 5、具备良好的质量意识，对负责的测试项目的质量负责，能独立承担项目质量分析和推动工作

任职要求：

- 1、2年以上移动APP/手机软件测试经验
- 2、熟悉Android和iOS平台主要机型和性能，能够制定测试规范和方案
- 3、有编程基础，熟悉Java、Python等编程语言
- 4、精通测试流程和测试方法，熟练掌握测试用例设计和编写方法
- 5、熟练自动化测试框架和工具，如Selenium、Appium、JMeter等
- 6、良好的学习能力、沟通能力和团队协作能力，具备较强的责任心和自我驱动力
- 7、熟悉主流测试工具，包括抓包工具、性能测试工具、自动化测试工具等

01

项目介绍

- 项目概述
- APP测试范围

学习目标

Learning Objectives

1. 熟悉被测项目类型，积累相关类型经验

01-项目介绍

- 项目：TPSHOP类似于淘宝、京东等电商平台
- 项目类型：电商平台
- 项目特点：
 - ① 业务通用性强
 - ② 更贴近生活，应用范围广



02-核心业务及模块

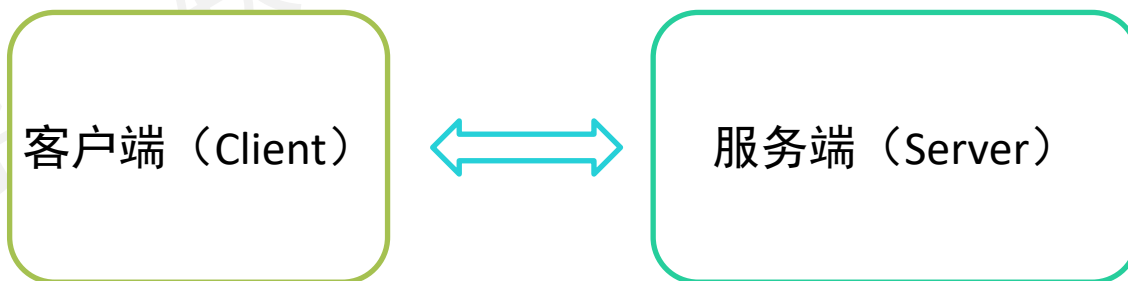
- 项目角色：买家、卖家等
- 项目业务：下单业务、发货业务等
- 系统及模块：

买家：注册登录、商品搜索、购物车、下单、支付、售后

卖家：会员管理、商品管理、订单管理、营销管理、物流管理、供应商管理

03-项目架构

- 系统架构
 - 开发语言
 - 前端：HTML+CSS+JavaScript
 - 后端：PHP
 - 应用服务器：Nginx、Apache
 - 数据库：Mysql
- 客户端：APP、web



前端: <https://hmshop-test.itheima.net/>

后端: <https://hmshop-test.itheima.net/admin>

总结

1. 项目介绍

① 项目类型：电商项目

② 主要功能：

买家：注册登录、商品搜索、购物车、下单、支付、售后

卖家：会员管理、商品管理、订单管理、营销管理、物流管理、
供应商管理

2. 项目技术栈

客户端：APP

服务端：Nginx+MySQL+PHP

架构：C/S架构

02

项目介绍

- 项目概述
- APP测试范围

01-APP测试和web测试对比

相同点

1. 同一项目APP和web使用的**后端服务器**是相同的
2. 前后端都使用**HTTP协议**进行交互（也有部分APP用socket来交互）

不同点

1. APP是**C/S**架构，web浏览器是**B/S**架构
2. APP前后端交互的数据格式以**Json**为主，web前后端交互的数据格式为**Json/HTML**等都有

备注

JSON(JavaScript Object Notation) 是一种轻量级的数据交换格式。采用完全独立于编程语言的文本格式来存储和表示数据。
JSON最常用的格式是对象的键值对。例如：{"Name": "张三", "age": 18}

02-APP测试范围

功能测试

业务测试
功能模块测试

性能测试

- CPU、内存占用
- 启动速度
- 流量、电量消耗
- 流畅度
- 稳定性

专项测试

- 安装卸载升级
- push消息推送
- 交叉事件测试
- 用户体验测试
- 兼容性测试



总结

1. APP测试和web测试对比

相同点：同一项目服务器相同

不同点：架构不同、服务器响应数据格式不同

2. APP测试范围

- ①APP功能
- ②APP专项
- ③APP性能

02

APP功能测试实施

- 业务测试
- 单模块测试
- 单模块测试案例



思考

1. 拿到一个项目一般是先测试单功能还是业务?

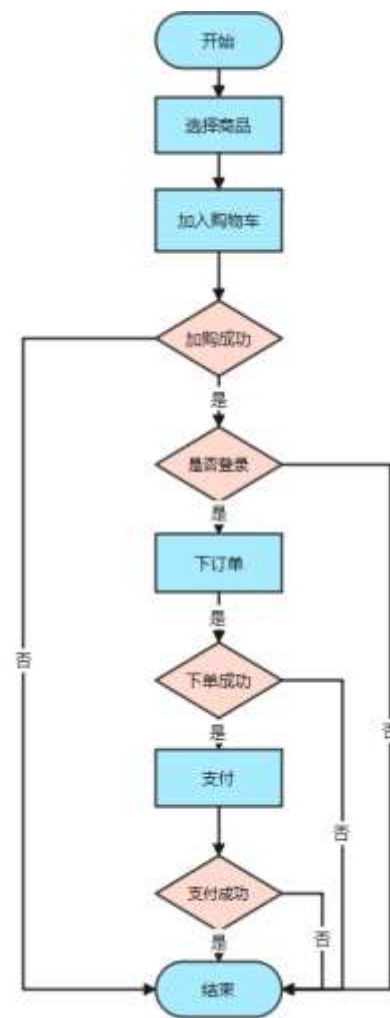
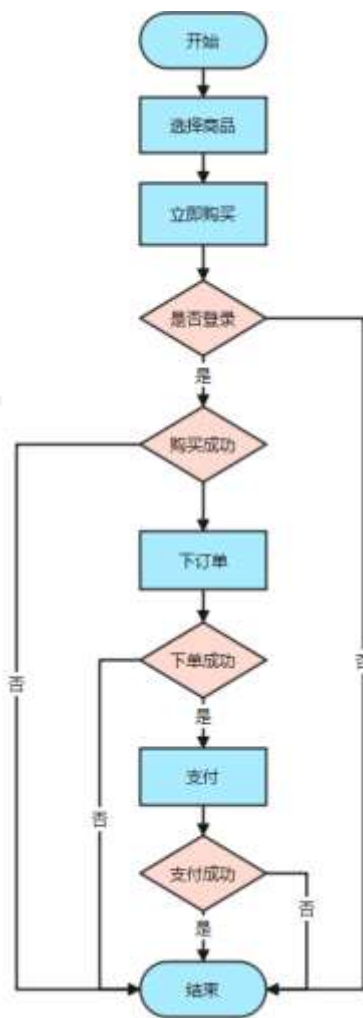
——业务（先要确保基本功能能够走通）

案例

针对tpshop“购物流程”设计测试用例

分析：

- ① 根据流程图找出业务路径
- ② 设计测试用例覆盖业务路径

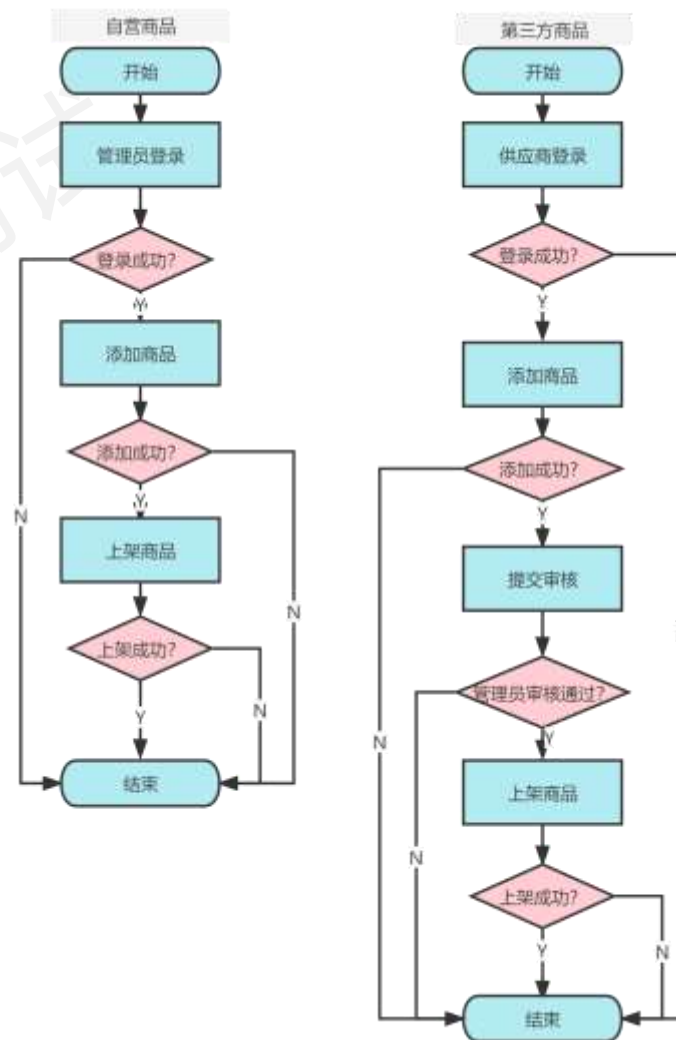


案例

针对tpshop“商品上架流程”设计测试用例

分析：

- ① 根据流程图找出业务路径
- ② 设计测试用例覆盖业务路径



总结

1. 拿到项目之后是先测试业务还是先测试单模块?

① 结论：优先测试核心业务，验证程序是否具备可测性。

② 用例优先级设置建议：

- P0：业务正向
- P1：业务逆向
- P2：单模块正向
- P3：单模块逆向

2. 如何使用业务流程设计用例

- ① 根据流程图找路径
- ② 根据路边编写用例

3. 业务流程设计注意事项

- ① 用例标题：看业务流程核心目的
- ② 前置条件：看菱形判断
- ③ 测试步骤：看长方形操作

02

项目测试实施

- 业务测试
- 单模块测试
- 单模块测试案例

01-APP功能测试

01

APP功能

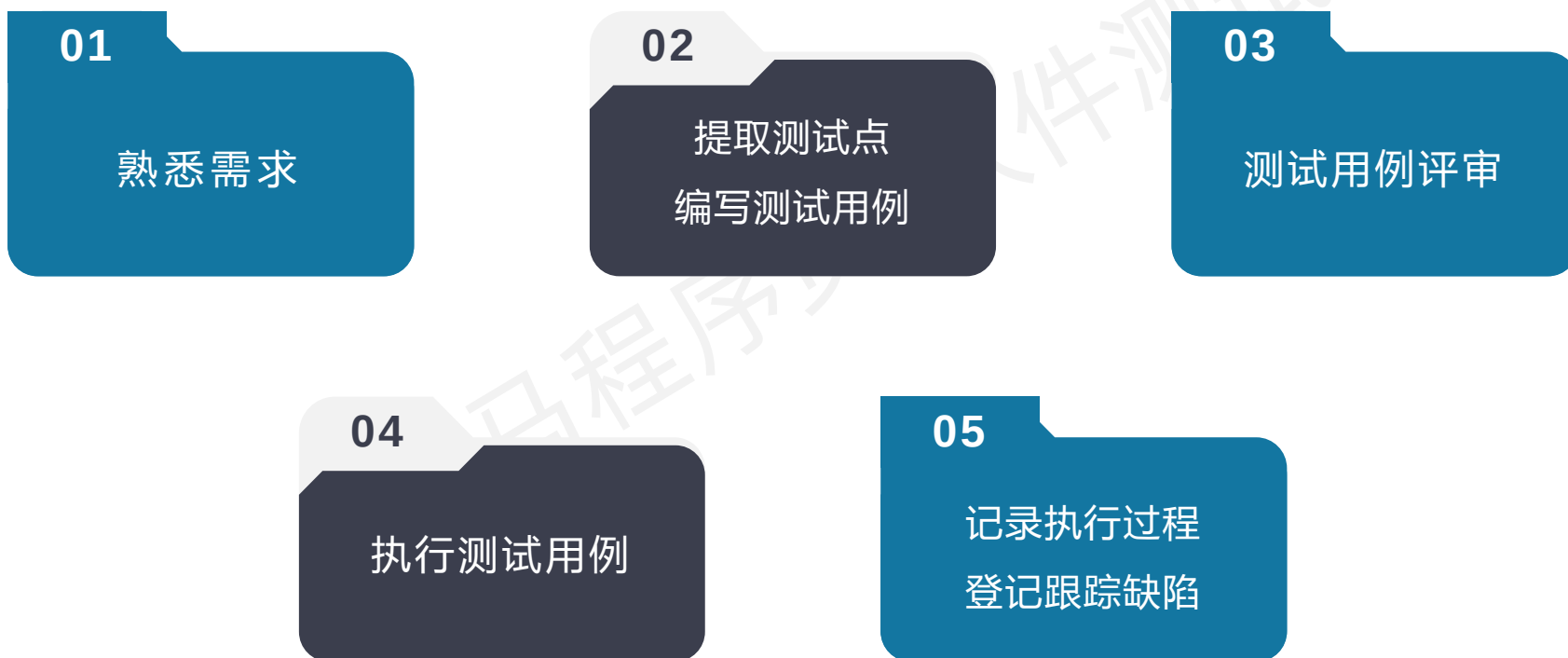
显示

操作

需求说明书

UI原型图

02-单模块测试步骤



03-测试点及用例设计思路

- 怎么开展单功能测试?

1. 熟悉需求：（核心）测试目的+条件

2. 梳理测试点

- ◆ 功能（能不能）：

- ①显示：是否正常

- ②操作（含规则）：能否成功

- ◆ 非功能（**后续单独测试**）：

- ①APP专项：APP特有测试项（在不同设备上稳定运行）

- ②APP性能：APP对移动端设备的影响程度（稳定性、资源）

3. 将测试点转化为测试用例

总结

1. 单模块设计范围

- APP功能（显示+操作）

2. 单模块测试步骤

- ① 熟悉需求
- ② 提取测试点、编写测试用例
- ③ 测试用例评审
- ④ 执行测试用例
- ⑤ 记录执行过程、登记跟进缺陷

02

项目测试实施

- 业务测试
- 单模块测试
- 单模块测试案例

案例

根据需求进行测试设计执行



账户名：格式正确，注册成功的用户账号（手机号或者邮箱）；

密码：注册成功账号对应密码。

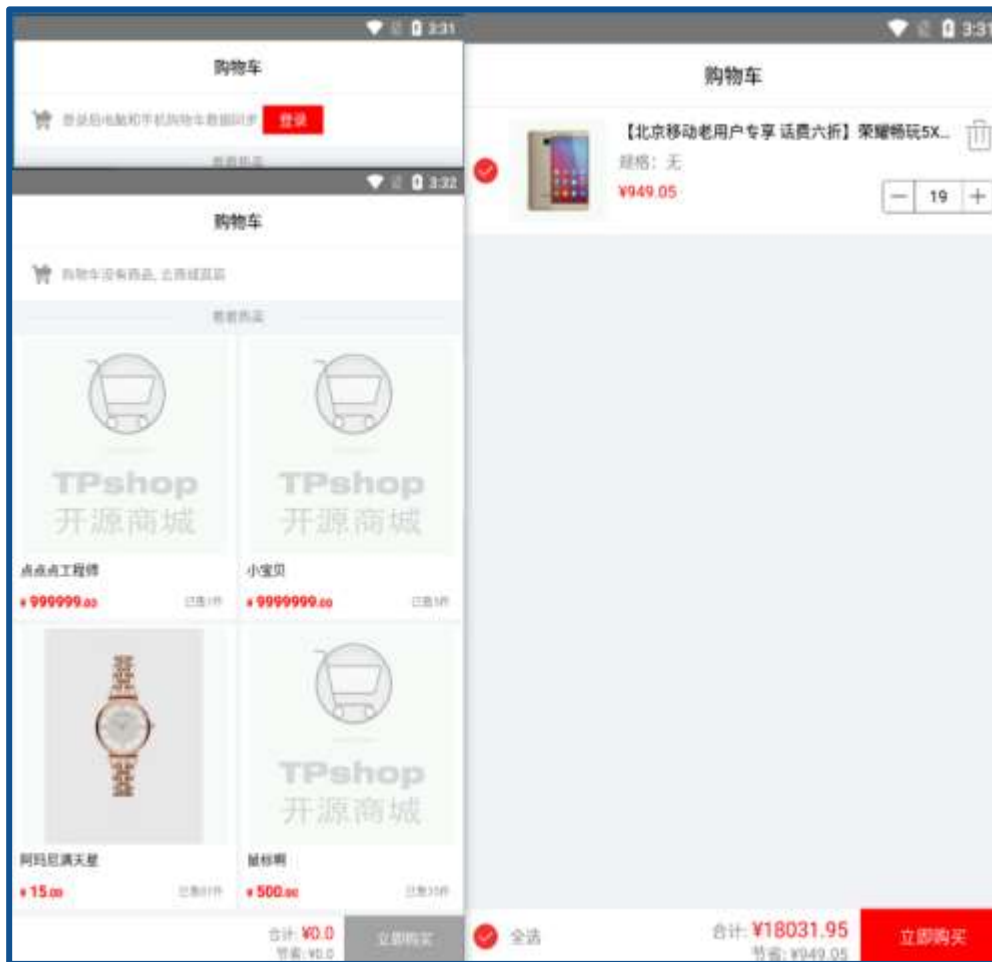
点击登录按钮，登录成功后跳转至个人中心，页面显示用户名。

登录失败显示对应提示。

详情参考需求文档登录模块

案例

针对购物车模块设计测试用例



1. 购物车显示

- 未登录提示去登录
- 已登录没有商品，提示马上去购物
- 已登录且有商品：
 - 待确认信息包含：商品名称、商品图片、单价、数量、小计、总价、已节省信息
 - 点击商品图片或名称跳转至商品详情页
 - 点击【去结算】跳转至填写核对订单页

2. 购物车添加商品

- 商品数量：
 - 库存 ≥ 200 时，商品数量取值为[1, 200];
 - 库存 < 200 时，商品数量取值为[1, 库存量]。
- 商品种类：商品种类不能大于20种

3. 购物车删除商品

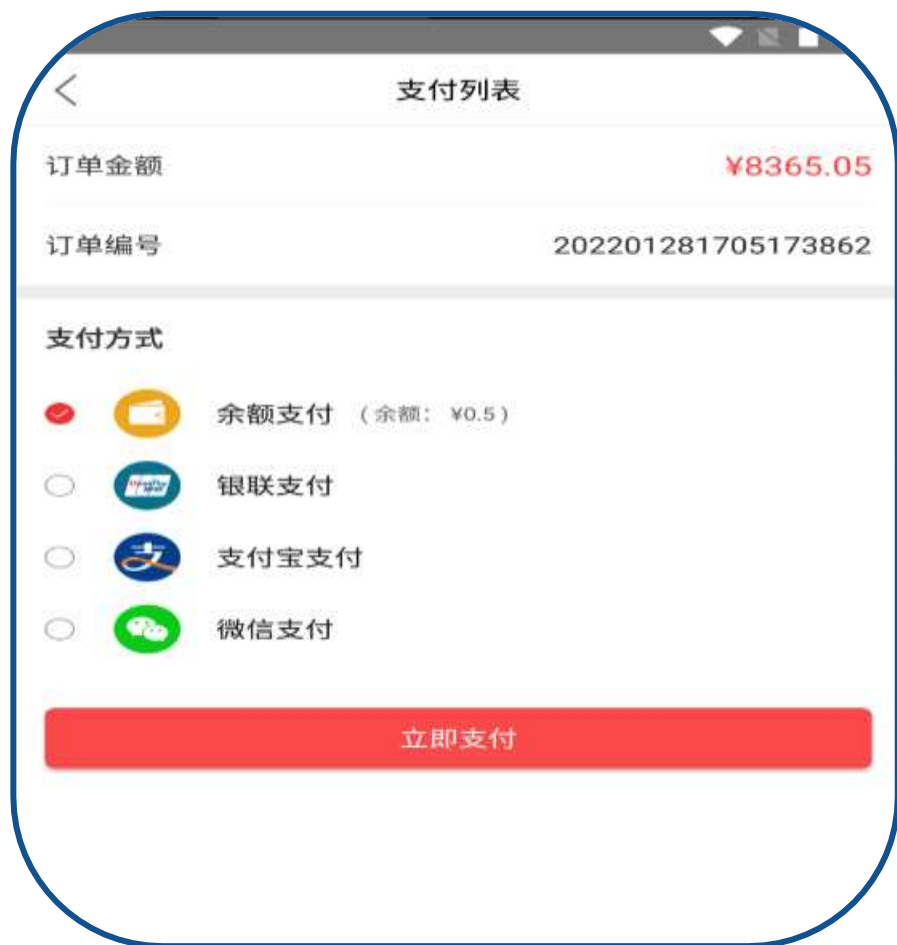
- 删除单个商品：点击对应商品后的删除符号

4. 购物车编辑商品购买数量：

- 编辑方式：
 - 使用【+】和【-】调节商品数量”；
 - 直接修改编辑框中的数字，修改商品数量；
- 编辑要求：参考购物车添加商品要求。

5. 结算

- 单件结算：勾选商品后点击立即购买
- 全选结算：全选后点击立即购买

 练习**针对支付模块设计测试用例****1. 支付方式说明**

- 银联支付：通过绑定网上银行卡进行在线支付
- 余额支付：通过账号余额进行在线支付
- 支付宝：通过支付宝APP在线支付
- 微信：通过微信APP在线支付

2. 支付显示操作

- 提交订单后默认选中余额支付
- 能够切换不同支付方式，不同支付方式不能同时选中
- 支付成功时，付款金额显示正确，提示：订单提交成功，我们将在第一时间给你发货！
- 选择银联支付确认提交后，能够打开网银页面并支持绑卡支付
- 选择余额确认提交后，直接从余额扣款
- 选择支付宝/微信提交后，能够跳转第三方APP进行支付

03

APP非功能测试实施

- APP专项测试
- APP性能测试

APP专项

- 说明：在不同的移动设备上能持久、稳定的运行App程序

1		抖音短视频	▲
2		微信	▼
3		淘宝	-
4		支付宝	▼
5		QQ	-
6		拼多多	▼
7		爱奇艺	▲
8		腾讯视频	▼
9		百度	▲
10		优酷视频	▼



环境

App应用运行所依赖的软硬件

- 依赖

1. mumu模拟器（移动设备）
2. App安装包



TPShop.apk



MuMu模拟器

mumu模拟器（移动设备）

说明：由网易编写一款安卓模拟器(window/mac)

win安装：右键管理员下一步安装完成即可。



APP专项测试范围

01

安装卸载升级

02

兼容性

03

push消息推送

04

交叉事件

05

用户体验

01

安装卸载升级

01-安装卸载升级

- 使用前：确保安装、升级功能正常

安装测试关注点：

- 正常场景：

- 来源：从**不同的安装渠道**安装（APP商城、手机助手、直接下载apk或者ipa文件安装）
- 去向：在**不同的操作系统版本**上安装
- 不同的安装路径（安装到手机上、安装到SD卡上）
- 卸载后安装
- 正在运行时覆盖安装

- 异常场景：

- 安装时出现异常（**关机、断网**），恢复后能否继续安装
- 安装时存储**空间不足**
- 安装时手动**取消后再次安装**
- **低版本覆盖安装高版本**

01-安装卸载升级

- 使用后：确保卸载功能正常

卸载测试关注点：

- 正常卸载（APP手动卸载、工具卸载）
- 运行时卸载
- 取消卸载
- 卸载异常中断
- **卸载后无数据残留**

升级测试关注点：

- 从**临近**版本升级
- **跨版本**升级
- **不同渠道**升级（应用商场、手机助手）
- **升级提醒**成功（可不提醒、可以提示升级、强制升级）
- 应用内升级时**非WIFI提醒**

注意事项

- **升级后要观察升级前的数据正常**（当数据结构改变而开发没有处理好时很容易出现升级前的数据混乱）

总结

1. 安装：系统版本+渠道+路径+异常后安装
2. 卸载：正常卸载+运行时卸载+取消后卸载
3. 升级：临近版本+跨版本+不同渠道

02

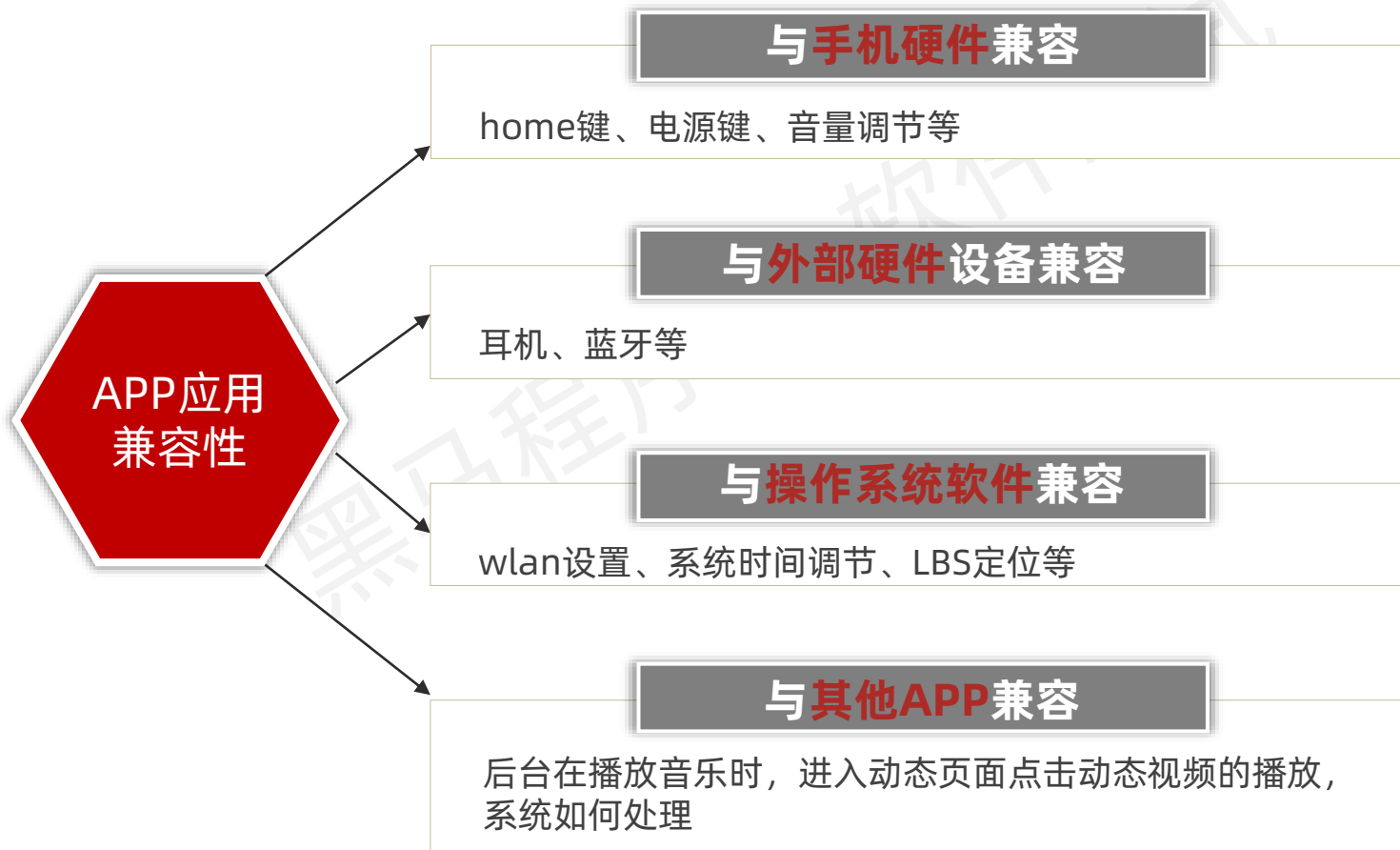
兼容性

02-APP兼容性

APP在不同的机型上由于**软件、硬件**等不同可能出现各种各样的问题，因此需要做兼容性测试

手机型号	覆盖市场主流机型（Android: vivo、小米、OPPO、华为） 考虑APP线上排名
系统版本	Android 系统：4.4、5.1、6.0、7.0 ... ios 系统：9.X、10.X、11.X、12.X...
分辨率 屏幕尺寸	分辨率 ：1080x1920、720x1280... 屏幕尺寸 ：5.5、4.7...
网络	2G、3G、4G、5G、 WIFI ...
应用 兼容性	手机硬件、外部硬件、操作系统、其他APP

02-APP兼容性



02-APP兼容性【扩展】

- 如何选择品牌、机型、系统、分辨率、网络？
 - 查找一些设备/应用排名网站（例如：<http://mi.talkingdata.com/terminals.html>）
 - 通过电商平台移动设备的销售量也可以预估



02-APP兼容性【扩展】

- 兼容性如何测试执行？
 - 一般情况下，在公司中会存在各种型号的手机，使用**真机**进行兼容性测试即可
 - 如果项目的用户量非常大，真机无法覆盖完全，可以**找第三方的兼容性平台**进行测试
 - 如：线上云测平台**testin** (<https://www.testin.cn/>)

总结

兼容性：品牌型号+分辨率+网络+软件+硬件

03

push消息推送

03-Push消息测试

- 什么是Push消息？
 - Push消息：是APP给你推送的各种消息。
 - 前提：需要在手机中“设置” - “通知”进行push消息的设置



03-Push消息测试

- Pull（拉）客户端主动获取：客户端固定时间主动向服务器获取消息。
- Push（推）客户端被动接受：当服务器有更新消息时，主动发送到客户端。

Pull方式消耗客户端和服务端资源

Push方式节省客户端和服务端资源

提示

- 在APP项目中，基于手机电量与流量的考虑，使用的都是push方式进行消息推送，因此又叫Push消息

03-Push消息推送流程



操作系统级别的消息推送服务器

- IOS: APNs (Apple Push Notification service)
- Android: C2DM (Cloud to Device Messaging)

自己搭建推送服务器

- 好处: 性能强、安全性高
- 缺点: 成本高



第三方推送平台

- 手机厂商类: 小米推送、华为推送。
- 第三方平台类: 友盟推送、极光推送、云巴 (基于MQTT)
- BAT大厂的平台推送: 阿里云移动推送、腾讯信鸽推送、百度云推送

03-Push消息测试关注点

Push消息推送的设置:

- APP服务器设置:
 - 推送内容
 - 推送时机
 - 推送频率
 - 推送人群（全部用户/部分用户）
- 手机端设置:
 - 是否接收通知
 - 提醒位置等



push消息测试关注点:

- APP服务器设置测试点:
 - Push消息是否按指定业务规则发送
 - 当Push消息是针对特定用户时，检查收到的Push与用户身份是否相符等
- 手机端设置测试点:
 - 设置不接收推送消息时，用户是否会收到Push消息
 - 设置push消息显示的位置，是否与配置一致
 - 收到push消息，是否能正常打开跳转等
- 其他测试:
 - APP在前台使用时，收到push消息如何提示
 - APP在后台运行时，收到push消息如何提示
 - APP离线，是否能收到PUSH消息。

总结

1. Push消息：app接收的各种通知
2. 推送服务器：操作系统级别+自己搭建+三方推送
3. 关注点：内容+业务规则+人群+显示/关闭通知+位置

04

交叉事件

05

用户体验

04-交叉事件测试

又叫（冲突、干扰）测试，是指一个功能正在执行过程中，另外一个事件或操作对该过程进行干扰的测试。

如：在App前台/后台运行同时接听来电或者下载文件等。

交叉事件测试关注点：

- APP运行时接打电话；
- APP运行时收发信息；
- APP运行时查看应用推送
- APP运行接上蓝牙设备
- APP运行时接收文件弹窗提醒
- APP运行时旋转屏幕
- APP运行时切换网络（4G、Wi-Fi）；
- App运行时使用相机、计算器等手机自带应用；
- App运行时电量告警、插拔充电器。

04-用户体验测试

以主观的角度去感知产品或服务的舒适、易用、友好亲切程度。



总结

1. 交叉测试：app应用使用过程中被其他操作干扰影响
2. 用户体验：UI界面+易用(导航、菜单、提示)+横竖屏

03

APP非功能测试实施

- APP专项测试
- APP性能测试

APP性能测试

测试app使用期间占用硬件资源（cpu、内存、流量、电量）使用情况。

分类

- ① App程序运行时占用手机硬件资源情况
- ② App稳定性

如何测试App（资源）性能？

说明：使用**工具**或命令进行测试

- 工具

SoloPi是一个无线的 Android 自动化工具，具备录制回放、性能测试等功能。

- 功能

性能测试：能够对CPU、内存与网络环境进行限制，复现应用在性能较差、网络环境不佳场景下的表现。

录制回放：能够将用户的操作记录下来，支持在各个设备上回放。

一机多控：操作一台主机设备来控制多台从机设备，进行重复冗杂的兼容性测试，能够极大提升兼容性测试的效率。

- 下载：<https://www.pgyer.com/solopi>

安装

独立安装的 SoloPi（APK，IOS无该版本），像普通APP一样安装。

SoloPi使用（选择测试项）

(1) 打开SoloPi，选择性能测试



(2) 选择被测应用，勾选监控指标，勾选后悬浮窗会出现在手机屏幕上



提示

SoloPi使用时，需要申请悬浮窗权限（adb权限、读写权限），根据弹窗提示进行授权。

SoloPi使用（监控）

(3) 点击开始监控，随后打开被测APP应用，开始测试



SoloPi使用（查看结果）

（4）查看数据采集结果



SoloPi使用（查看结果）

结果展示



可下拉选择其他性能项





总结

1. App性能测试分类:

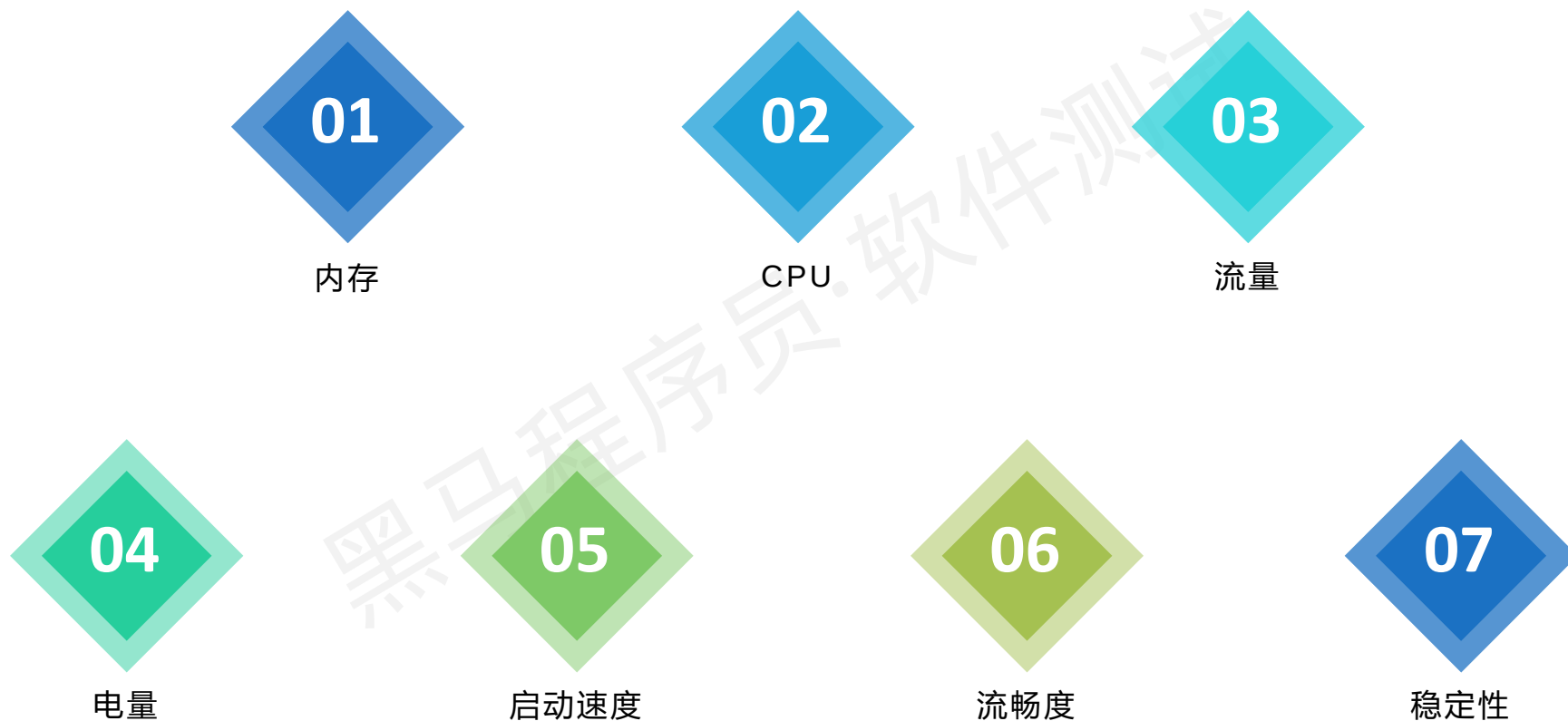
资源占用 + 稳定性

2. App性能工具:

- ① **Android: 工具 (solopi、GT) + 命令 (adb)**
- ② **Ios: 苹果开发工具xcode**

APP性能测试关注点

- APP使用时对CPU、内存的占用情况;
- APP使用时是否流畅等;
- APP使用时电量流量的消耗情况;
- APP的启动时间是否过长;
- APP是否能长时间稳定运行



01

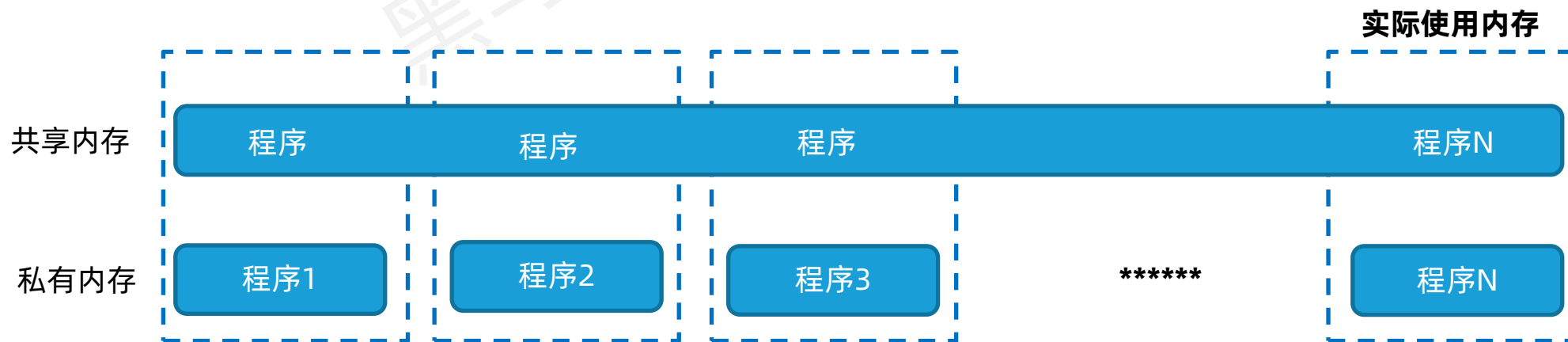
内存

内存监控指标

每个程序运行时都需要将代码和数据放入内存中，内存不足则程序无法正常运行。

提示：SoloPi工具提供了两个内存的监控指标：Private dirty 和 PSS。

- Private dirty（私有内存）：
 - 进程独占内存，也就是进程销毁时可以回收的内存容量。
- PSS（实际使用内存）：
 - 将跨进程共享页也加入进来，进行按比例计算PSS。这样能够比较准确的表示进程占用的实际物理内存。



内存问题的现象

常见的现象：

- 内存泄漏：
 - 内存泄露 memory leak，是指程序在申请内存后，无法释放已申请的内存空间，一次内存泄露危害可以忽略，但内存泄露堆积后果很严重，无论多少内存,迟早会被占光。
 - memory leak会最终会导致内存溢出。
- 内存溢出：
 - 内存溢出 out of memory，是指程序在申请内存时，没有足够的内存空间供其使用，出现out of memory

内存问题产生的影响：

- 程序实际使用的内存PSS持续增长可能是内存泄漏

已使用：281,346,208 个字节



- 程序出现crash（崩溃）可能是内存溢出

案例

性能内存测试

- 需求：浏览tpshop首页平均内存消耗情况。
- 步骤：
 - (1) 打开SoloPi工具，配置内存监控
 - (2) 进入APP，操作上述业务，观察运行时的内存指标
 - (3) 查看内存运行结果
 - 检查程序实际使用的内存PSS值



总结

1. 性能:

- ① 内存 + cpu + 流量 + 电量
- ② 启动速度 + 流畅度 + 稳定性

2. 内存关注:

- ① 实际使用内存 (PSS)
- ② 私有内存

3. 内存常见问题:

- ① 内存泄漏: 申请内存无释放内存。
- ② 内存溢出: 申请内存时, 无内存可用。

02

CPU

CPU监控指标

SoloPi工具提供了两个CPU的监控指标：全局占用CPU和应用进程CPU。

- 全局占用CPU：整机的CPU使用水平，即当前手机的CPU整体使用率。
 - 在 Linux 系统下，CPU 利用率分为用户态、系统态和空闲态
 - 用户态：表示 CPU 处于应用程序执行的时间
 - 系统态：表示系统内核执行的时间
 - 空闲态：表示空闲系统进程执行的时间。
 - CPU 使用率 = CPU 执行非系统空闲进程时间 / CPU 总的执行时间
- 应用进程CPU：表示自开机以来，应用程序消耗的CPU时间的总数。

CPU消耗引起的现象

- CPU使用长时间处于90%以上
- 手机发热、耗电量增加
- 响应变慢、引起ANR (Application Not Responding)



案例

性能CPU测试

需求：测试滑动首页CPU使用率

步骤：

- (1) 打开SoloPi工具，勾选CPU监控指标
- (2) 进入tpshop，操作上述业务，观察运行时的CPU指标
- (3) 查看CPU运行结果
 - 检查APP运行时CPU是否长时间处于90%以上



总结

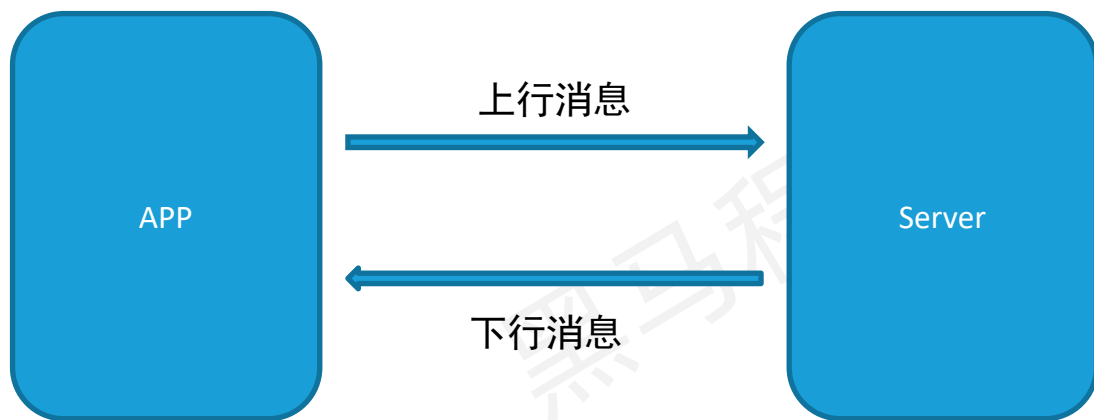
1. 关注：长时间90%以上
2. cpu问题现象：手机发热 + 耗电量增加 + 反应变慢 + 无响应

03

流量

流量介绍

操作APP会与服务器交换数据，流量就是指这些交互数据的总大小。



SoloPi工具提供了流量监控指标：网络

提示

上行消息是APP发送给服务器的数据，下行消息是APP接收的服务器的数据

**案例****性能流量测试**

需求：打开tpshop首页，上下滑动动态20秒，获取消耗的网络流量。

步骤：

- (1) 打开SoloPi工具，勾选流量监控指标网络
- (2) 进入APP，操作上述业务
- (3) 查看流量统计结果

提示：在模拟器中无法统计电脑流量使用情况，看进程使用流量即可

全局下行速率 - Network

累计全局上行流量 - Network

累计全局下行流量 - Network

进程上行流量-main-19868 - Network

进程上行速率-main-19868 - Network

进程下行流量-main-19868 - Network

进程下行速率-main-19868 - Network



总结

流量优化策略:

- 数据的压缩
- 不同数据格式的采用
- 控制访问的频次
- 只获取必要的数据
- 缓存机制
- 针对不同的网络类型设置不同的访问策略

04

电量

电量

APP应用使用时对电池电量的平均消耗

常见的耗电量大的场景：

- 定位
- 网络传输
- 屏幕亮度
- wake_locker (锁屏-解锁)

电量的监控方法

- 系统自带接口
 - 最新的IOS和Android系统内置的Setting里可以查看各个APP的电池消耗
 - 该方案不能检测固定某一段时间内的电池精准消耗。
- 硬件检测
 - 通过硬件可以精准地获得应用的电量消耗（如：PowerMonitor 硬件设备）
 - 该方案测试时需要拆机，成本太高比较麻烦。
- 软件工具检测
 - 通过第三方的软件来获取应用的电量消耗（如：AccuBattery、360省电王、SoloPi等）
 - 该方案取决于第三方软件的计算准确性。
- 结果分析
 - 与基准数据对比（基准数据来自于产品经理，或者以往数据积累）
 - 横向对比（竞品）（目前多数采用这种方法）



SoloPi工具提供了电量的监控指标：电池。

案例

性能电量测试

- 需求：打开Tpshop，进入首页，上下滑动动态2分钟，获取消耗的电量。
- 步骤：
 - (1) 打开SoloPi工具，勾选电量监控指标：电池
 - (2) 进入APP，操作上述业务，观察运行时的CPU指标
 - (3) 保存电量详细数据后，可以查看电量详细的数据统计。

提示：模拟器没有电池，无法获取电量数据。



总结

1. 耗电量大场景:

- 定位
- 网络传输
- 屏幕亮度
- wake_locker (锁屏-解锁)

2. 分析结果:

- 与基准数据对比 (基准数据来自于产品经理, 或者以往数据积累)
- 横向对比 (竞品)

05

流畅度

流畅度介绍

动画播放或图片切换的流畅性

动画其实是由一张张画出来的图片连贯执行产生的效果，当一张张独立的图片切换速度足够快的时候，会欺骗我们的眼睛，以为这是连续的动作。



反之，当图片切换不够快时就会被人眼看穿，反馈给用户的就是卡顿现象



流畅度的监控指标

SoloPi工具提供了流畅度的监控指标：帧率FPS

- 即Frames per second：GPU在一秒内绘制的帧数。（简单理解为一秒内呈现给用户的图片数）
- FPS值越高画面越流畅

流畅度问题产生的影响：

- 想要让大脑觉得动作是连续的，至少是每秒10-12帧的速度
- 想达到流畅的效果，至少需要每秒24帧
- 60帧每秒的流畅度是最佳的，我们的目标就是让程序的流畅度能接近60帧每秒

注意事项

- 当页面多为静态时，FPS值很小是正常的
- 页面数据多为动态加载时，FPS值比较大（建议在24帧以上）

案例

性能流畅度测试案例

需求：打开tpshop，进入首页，上下滑动动态2分钟，记录FPS值。

步骤：

- (1) 打开SoloPi工具，勾选帧率
- (2) 进入APP，操作上述业务，观察运行时的流畅度指标
- (3) 查看流畅度运行结果

总结

流畅度：动画播放或图片切换的流畅性

- 达到流畅效果24/S
- 最佳流畅效果60/S

06

启动速度

启动速度介绍

APP启动速度：从启动app到主页面加载完成的速度。

APP启动分类：冷启动、热启动

- 冷启动：启动app进程，这种启动方式叫做冷启动。
- 热启动：将app从后台置于前台。

Solopi指标：启动耗时计算

案例

性能启动时间测试

需求：分别获取tpshop冷启动和热启动时间

黑马程序员·软件测试

总结

启动分类：热启动+冷启动

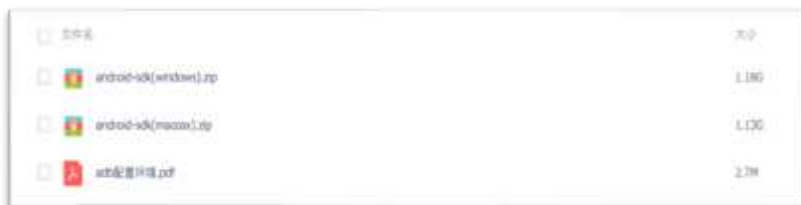
- 冷启动：启动app进程，这种启动方式叫做冷启动。
- 热启动：从后台切换到前台

Android-sdk环境说明

android开发、调试工具包

- 作用：android应用稳定性测试、调试工具、日志记录等
- 下载：<https://pan.baidu.com/s/15cQCOazgrmGUYMDiRKQxyA?pwd=9gbj>

- 安装：



1、解压到指定目录 如 (D:\Android\sdk)

2、将目录添加到path中

- ① 新建环境变量：ANDROID_HOME=D:\Android\sdk (这里为安装目录)
- ② 添加path路径，在Path中添加：%ANDROID_HOME%\tools、%ANDROID_HOME%\platform-tools

- 验证：win:打开开始菜单->运行->输入cmd->adb version

```
lgy@yourhostname ~ % adb version
Android Debug Bridge version 1.0.39
Version 0.0.1-4500957
Installed as /Users/lgy/Documents/传智/环境/移动自动化环境/mac/android-sdk-macosx/platform-tools/adb
```

07

稳定性

在使用app过程中，是否遇到以下现象？



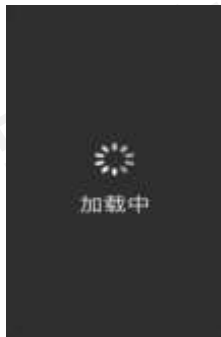
崩溃



闪退



无响应



卡顿



什么是稳定性?

app程序能持久良好的运行。



稳定性测试：在app应用中随意操作，挖掘有可能出现的异常。

卡顿
闪退 崩溃
无响应

Monkey介绍

monkey就是猴子， monkey测试就像一只猴子在玩手机（乱抓、乱点）。



作用：模拟用户随机（触摸屏幕、滑动、 按键）等操作来对程序进行稳定性测试， 检测程序异常情况。

提示

- Monkey程序是Android系统中自带一款稳定性测试工具，由Java语言编写。【无需单独安装】
- Android位置：/system/framework/monkey.jar

稳定性测试步骤

1、执行命令，执行结果写入日志

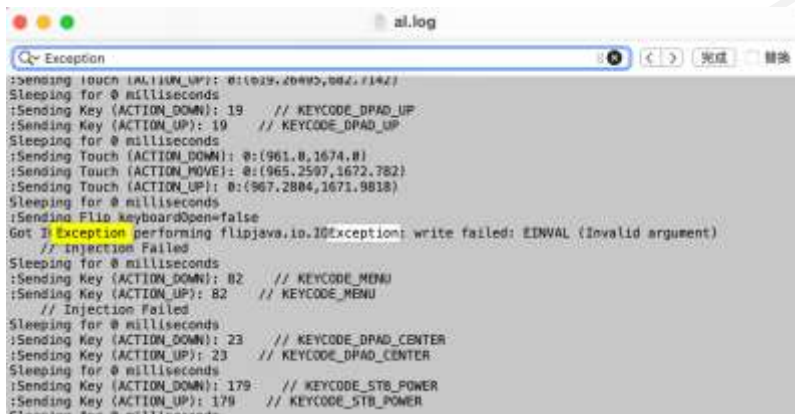
```
adb shell monkey -p com.tpshop.malls -v 2000 > tpshop.log
```

包名

次数

结果文件

2、检查日志异常



```
Exception
adb logcat -s
:Sending Touch (ACTION_UP): 0:(961.0,1674.0)
:Sleeping for 0 milliseconds
:Sending Key (ACTION_DOWN): 19 // KEYCODE_DPAD_UP
:Sending Key (ACTION_UP): 19 // KEYCODE_DPAD_UP
:Sleeping for 0 milliseconds
:Sending Touch (ACTION_DOWN): 0:(961.0,1674.0)
:Sending Touch (ACTION_MOVE): 0:(965.2507,1672.782)
:Sending Touch (ACTION_UP): 0:(967.2804,1671.9818)
:Sleeping for 0 milliseconds
:Sending Flip keyboardOpen=false
Got 7: Exception performing flipjava.io.IOException: write failed: EIO (Invalid argument)
// Injection Failed
:Sleeping for 0 milliseconds
:Sending Key (ACTION_DOWN): 82 // KEYCODE_MENU
:Sending Key (ACTION_UP): 82 // KEYCODE_MENU
// Injection Failed
:Sleeping for 0 milliseconds
:Sending Key (ACTION_DOWN): 23 // KEYCODE_DPAD_CENTER
:Sending Key (ACTION_UP): 23 // KEYCODE_DPAD_CENTER
:Sleeping for 0 milliseconds
:Sending Key (ACTION_DOWN): 179 // KEYCODE_STB_POWER
:Sending Key (ACTION_UP): 179 // KEYCODE_STB_POWER
:Sleeping for 0 milliseconds
```

Monkey命令

```
adb shell monkey -p com.tpshop.malls -v 2000 > tpshop.log
```

包名

次数

结果文件

语法: adb shell monkey -p 包名 -v 次数 > tpshop.log

参数:

-p:指定包名

-v:log详细程度 (最高支持' -v -v -v' 最详细)

次数:要执行随机操作的次数

>:重定向 (保存) 日志

提示: 获取包名: adb shell dumpsys window | findstr usedApp

```
C:\Users\zhujg>adb shell dumpsys window | findstr usedApp  
mFocusedApp=ActivityRecord{167c328 u0 com.tpshop.malls/.SPMainActivity t19}
```

检查日志

检查日志中是否有异常关键字，提取相关日志发给开发。

常见关键字：

- 无响应：在日志中搜索 “ANR” 、 timeout
- 崩溃：在日志中搜索 “NullPointerException” 或 Exception
- 闪退：memory out 、 memory Leak
- 错误：error、Err

提示：

- 1、测试人员一般不需要分析错误日志原因，如果具备日志分析能力可以辅助开发定位缺陷原因。
- 2、日志错误类型原因有很多，需要经验积累，以上关键字提供了常见错误关键字

案例

tpshop稳定性测试

需求：针对tpshop商城进行稳定性测试

分析：

- ① 获取app包名: com.tpshop.malls
- ② 执行monkey命令: adb shell monkey -p 包名 -v 次数 >c:\日志.log
- ③ 检查日志异常关键字: [ANR,timeout,Exception,out,leak,error]
- ④ 将异常日志发给开发

扩展

分类	参数	释义
基础参数	-v	指定反馈信息级别（信息级别就是日志的详细程度），总共分3个级别。最高级别为-v -v -v
	-S	指定monkey测试的序列
	-p	指定一个或多个包（Package，即App）
	--throttle	指定用户操作（即事件）间的时延，单位是毫秒
事件类型	--pct-touch	调整触摸事件的百分比(触摸事件是一个down-up事件，它发生在屏幕上的某单一位置)
	--pct-motion	调整动作事件的百分比(动作事件由屏幕上某处的一个down事件、一系列的伪随机事件和一个up事件组成)
	--pct-trackball	调整轨迹事件的百分比(轨迹事件由一个或几个随机的移动组成，有时还伴随有点击)
	--pct-syskeys	调整“系统”按键事件的百分比(这些按键通常被保留，由系统使用，如Home、Back、Start Call、End Call及音量控制键)
	--pct-anyevent	调整其它类型事件的百分比。它包罗了所有其它类型的事件，如：按键、其它不常用的设备按钮、等等
调试选项	--ignore--crashes	应用程序崩溃
	--ignore-timeouts	无响应ANR
	--igone-security-exceptions	许可证书崩溃
	--kill-process-after-error	发生错误停止运行并保持当前状态
	--montitor-native-crashes	监视并报告Androids系统本地代码的崩溃事件

总结

1. 常见app不稳定的现象

崩溃、闪退、无响应、卡顿

2. 稳定性测试

在app应用中随意操作，挖掘有可能出现的异常。

3. 稳定性怎么测？

- ① Android系统使用自带monkey工具
- ② 测试次数【工作中50W-80W次】

4. Monkey工具应用

- ① 执行Monkey命令：`adb shell monkey -p 包名 -v 次数 > 日志.log`
- ② 检查日志是否有异常，有异常将日志发给开发
[ANR,timeout,Exception,out,leak,error]



传智教育旗下高端IT教育品牌